Kubernetes YAML Files — Real-Time Examples

1. deployment.yaml
deployment.yaml main application ni Kubernetes lo run/manage cheyyadaniki use chestham. Ee file lo mana Docker image ni specify chestham, enni Pods kavalo replicas tho cheptham. Deployment desired state ni maintain chestundi, ante manam 3 Pods kavali ani chepthe Kubernetes 3 Pods running lo undela manage chestundi. Oka Pod fail ayithe replacement Pod create cheyyadaniki Kubernetes controllers work chestayi. Prathi Pod lopala containers section lo specify chesina Docker image tho container run avtundi.

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: netflix

spec:
  replicas: 3

  selector:
    matchLabels:
      app: netflix

  template:
    metadata:
      labels:
        app: netflix

    spec:
      containers:
        - name: netflix
          image: mycompany/netflix:v1
          ports:
            - containerPort: 80
```

Ikkada replicas: 3 → 3 Pods maintain cheyyali. containers → prathi Pod lo container configuration. image → container run cheyyadaniki use chese Docker image. containerPort: 80 → application container lo application listen chestunna port.
Flow ila untundi:

```
deployment.yaml
   ↓
Deployment
   ↓
ReplicaSet
   ↓
3 Pods
   ↓
┌──────────┬──────────┬──────────┐
│  Pod 1   │  Pod 2   │  Pod 3   │
│     │    │     │    │     │    │
│Container │Container │Container │
│  ↓  │    │  ↓  │    │  ↓  │    │
│netflix:v1│netflix:v1│netflix:v1│
└──────────┴──────────┴──────────┘
```

Example YAML lo:

```
spec:
  replicas: 3

  template:
    spec:
      containers:
        - name: netflix
          image: mycompany/netflix:v1
```

Ikkada replicas: 3 → 3 Pods. containers: → prathi Pod lo container configuration. image: → aa container lo run cheyyalsina Docker image.
So simple ga:

```
Deployment
   ↓
Pods create/manage
   ↓
Prathi Pod lo Container
   ↓
Container lo netflix:v1 image
   ↓
```

Netflix application

---

2. service.yaml

service.yaml already running unna Pods ki **stable network endpoint** provide cheyyadaniki use chestham. Pods ki individual IP addresses change avvachu, kabatti users or other applications direct ga Pod IP ni use cheyyakunda Service ni access chestayi. Service lo selector petti ye Pods ki traffic vellalo cheptham. Ee file Pods or containers create cheyyadu; Deployment create/manage chesina Pods ki traffic route chestundi.

```
apiVersion: v1
kind: Service

metadata:
  name: netflix-service

spec:
  selector:
    app: netflix

  ports:
    - port: 80
      targetPort: 80

  type: ClusterIP
```

Ikkada selector: app: netflix → app: netflix label unna Pods ni select chestundi. port: 80 → Service expose chese port. targetPort: 80 → traffic vellalsina Pod/container port. type: ClusterIP → Service cluster lopala access cheyyadaniki.

Flow ila untundi:

```
service.yaml
    ↓
Netflix Service
    ↓
selector: app=netflix
    ↓
┌──────────┬──────────┬──────────┐
│  Pod 1   │  Pod 2   │  Pod 3   │
│Container │Container │Container │
└──────────┴──────────┴──────────┘
```

Example YAML lo:

```
spec:
  selector:
    app: netflix

  ports:
    - port: 80
      targetPort: 80
```

selector → ye Pods ki traffic pampalo identify chestundi. port → Service port. targetPort → selected Pod lo application port.

So simple ga:

```
User / Application
    ↓
Netflix Service :80
    ↓
Matching Netflix Pods
    ↓
Containers
    ↓
Netflix Application
```

---

3. ingress.yaml

ingress.yaml external users nunchi vachina **HTTP/HTTPS requests ni correct Service ki route cheyyadaniki** use chestham. User browser lo https://netflix.example.com open chesthe, domain/path based ga request ni netflix-service ki pampinchadaniki Ingress rules use avtayi. Ee file direct ga Pods create cheyyadu, Service kuda create cheyyadu; already unna Service ki external traffic route cheyyadaniki configuration istundi. Real-time lo Ingress ni process cheyyadaniki cluster lo Ingress Controller undali.

```
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: netflix-ingress

spec:
  rules:
    - host: netflix.example.com

      http:
```

```
    paths:
     - path: /
       pathType: Prefix

       backend:
        service:
          name: netflix-service
          port:
            number: 80
```

Ikkada host: netflix.example.com → user access chese domain. path: / → request URL path. service.name: netflix-service → request vellalsina Service. number: 80 → Service port.

Flow:

```
ingress.yaml
   ↓
Ingress
   ↓
https://netflix.example.com
   ↓
netflix-service
   ↓
Netflix Pods
   ↓
Containers
   ↓
Netflix Application
```

Example:

```
User Browser
   ↓
https://netflix.example.com
   ↓
Ingress
   ↓
netflix-service:80
   ↓
Pod 1 / Pod 2 / Pod 3
```

So simple ga:

Ingress → External request ni Service ki route chestundi.

---

4. configmap.yaml

configmap.yaml application ki kavalsina **normal configuration values** ni separate ga maintain cheyyadaniki use chestham. Application code or Docker image lo configuration hard-code cheyyakunda ConfigMap nunchi values provide cheyyachu. Development, staging, production environments ki different configuration values use cheyyadaniki idi useful. ConfigMap itself Pods or containers create cheyyadu; existing application ki configuration provide chestundi.

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: netflix-config

data:
  APP_ENV: production
  API_URL: https://api.netflix.example.com
  LOG_LEVEL: info
```

Ikkada APP_ENV → application environment. API_URL → backend API URL. LOG_LEVEL → application logging level.

Deployment lo ee ConfigMap values container ki environment variables ga ivvachu:

```
spec:
  template:
    spec:
      containers:
       - name: netflix
         image: mycompany/netflix:v1

         envFrom:
          - configMapRef:
              name: netflix-config
```

Flow:

```
configmap.yaml
   ↓
ConfigMap
   ↓
Deployment
```

↓
Container
↓
Application Configuration
Example:
ConfigMap
APP_ENV=production
API_URL=...
LOG_LEVEL=info
↓
Netflix Container
↓
Netflix Application
So simple ga:
ConfigMap → Application ki normal configuration provide chestundi.

---

5. secret.yaml
secret.yaml passwords, API keys, tokens, database credentials lanti **sensitive configuration values** kosam use chestham. Ee values ni application code lo direct ga hard-code cheyyakunda Kubernetes Secret nunchi container ki provide cheyyachu. Secret kuda Pods or containers create cheyyadu; application ki sensitive configuration provide chestundi. Real production lo Secrets ni proper access control and encryption practices tho manage cheyyali.

```
apiVersion: v1
kind: Secret

metadata:
  name: netflix-secret

type: Opaque

stringData:
  DB_USERNAME: netflix
  DB_PASSWORD: example-password
```

Ikkada DB_USERNAME → database username. DB_PASSWORD → database password. stringData lo readable string values provide chestunnam.
Deployment lo Secret ni container ki provide cheyyachu:

```
spec:
  template:
    spec:
      containers:
        - name: netflix
          image: mycompany/netflix:v1

          envFrom:
            - secretRef:
                name: netflix-secret
```

Flow:
secret.yaml
↓
Secret
↓
Deployment
↓
Container
↓
Netflix Application
↓
Database
So simple ga:
Secret → Passwords/API keys lanti sensitive configuration kosam.

---

6. hpa.yaml
hpa.yaml application traffic/load periginaappudu **Pods automatic ga increase/decrease cheyyadaniki** use chestham. HPA direct ga Docker containers create cheyyadu; already unna Deployment ni target chesi replica count ni automatically adjust chestundi. Example ga minimum 2 Pods, maximum 10 Pods ani set cheyyachu. CPU usage configured target kanna ekkuva ayithe HPA additional Pods create avvadaniki Deployment ni scale chestundi.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: netflix-hpa
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: netflix

  minReplicas: 2
  maxReplicas: 10

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Ikkada scaleTargetRef → ye Deployment ni scale cheyyalo chepthundi. minReplicas: 2 → minimum 2 Pods. maxReplicas: 10 → maximum 10 Pods. averageUtilization: 70 → CPU target.

Flow:

```
hpa.yaml
   ↓
HPA
   ↓
Netflix Deployment
   ↓
Replica count change
   ↓
Pods increase/decrease
```

Example:

```
Normal Traffic
   ↓
2 Pods

High Traffic
   ↓
5 Pods

Very High Traffic
   ↓
10 Pods
```

So simple ga:

HPA → Traffic/load batti Deployment Pods ni automatically scale chestundi.

---

7. pvc.yaml

pvc.yaml application ki **persistent storage kavali** ani Kubernetes ki request cheyyadaniki use chestham. Pod recreate ayina important data survive avvali ante persistent storage use chestham. PVC lo entha storage kavalo and access mode enti ani specify chestham. PVC actual storage create cheyyadam kanna, storage kosam application/deployment chese request laga understand chesko.

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: netflix-storage

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 10Gi
```

Ikkada accessModes: ReadWriteOnce → storage access mode. storage: 10Gi → 10 GiB storage request chestunnam.

Deployment lo PVC ni container ki mount cheyyachu:

```
spec:
  template:
    spec:
      containers:
        - name: netflix
          image: mycompany/netflix:v1

          volumeMounts:
```

```
        - name: netflix-data
          mountPath: /data

    volumes:
      - name: netflix-data
        persistentVolumeClaim:
          claimName: netflix-storage
```

Flow:

pvc.yaml
↓
PVC
↓
Persistent Storage
↓
Pod
↓
Container
↓
/data

So simple ga:

PVC → Application ki persistent storage request/provide cheyyadaniki.

---

8. namespace.yaml

namespace.yaml Kubernetes cluster lo applications/environments ni **logical ga separate cheyyadaniki** use chestham. Example ga same cluster lo netflix-dev, netflix-test, netflix-prod separate ga maintain cheyyachu. Namespace itself application Pods ni create cheyyadu; Deployment, Service, ConfigMap etc. resources ekkada belong avvalo organize cheyyadaniki use chestham.

```
apiVersion: v1
kind: Namespace

metadata:
  name: netflix-prod
```

Ikkada name: netflix-prod → Namespace name.

Deployment ni ee Namespace lo create cheyyalante:

```
metadata:
  name: netflix
  namespace: netflix-prod
```

Service kuda:

```
metadata:
  name: netflix-service
  namespace: netflix-prod
```

Flow:

namespace.yaml
↓
netflix-prod Namespace
↓

```
Deployment
Service
Pods
ConfigMap
Secret
HPA
```

So simple ga:

Namespace → Kubernetes resources ni logical ga separate/organize cheyyadaniki.

---

9. serviceaccount.yaml

serviceaccount.yaml Pod ki **Kubernetes identity** provide cheyyadaniki use chestham. Application Pod Kubernetes API tho interact avvalsina situation lo ServiceAccount useful. Example ga application ki Kubernetes lo Pods ni read cheyyadaniki permission ivvali anukunte ServiceAccount + Role + RoleBinding use chestham. ServiceAccount create chesinanta matrana permissions automatically ravu.

```
apiVersion: v1
kind: ServiceAccount

metadata:
  name: netflix-sa
```

Deployment lo ServiceAccount ni assign cheyyachu:

```
spec:
  template:
    spec:
      serviceAccountName: netflix-sa
```

```
    containers:
     - name: netflix
       image: mycompany/netflix:v1
```
Flow:
serviceaccount.yaml
↓
Netflix ServiceAccount
↓
Netflix Pod
↓
Kubernetes API
So simple ga:
ServiceAccount → Pod/Application ki Kubernetes identity provide chestundi.

---

10. role.yaml
role.yaml Kubernetes resources meeda **ye permissions kavalo define cheyyadaniki** use chestham. Example ga Netflix application ki Pods ni read cheyyadaniki permission kavali kani Pods delete cheyyadaniki permission avasaram ledu anukundam. Appudu Role lo get and list matrame define chestham. Role itself permission evariki ivvalo cheppadu; danikosam RoleBinding use chestham.
```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role

metadata:
  name: netflix-role

rules:
  - apiGroups:
      - ""
    resources:
      - pods
    verbs:
      - get
      - list
```
Ikkada resources: pods → Pods meeda permissions. get → Pod details read cheyyachu. list → Pods list chudachu. delete permission ledu.
Flow:
role.yaml
↓
netflix-role
↓
get + list Pods
So simple ga:
Role → "E permissions kavali?" ani define chestundi.

---

11. rolebinding.yaml
rolebinding.yaml lo Role lo define chesina permissions **evariki assign cheyyalo** cheptham. Role get/list Pods permission define chesthe, RoleBinding aa permission ni netflix-sa ServiceAccount ki assign chestundi. So Role and RoleBinding rendu kalisi work chestayi. RoleBinding itself Pods or containers create cheyyadu.
```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding

metadata:
  name: netflix-rolebinding

subjects:
  - kind: ServiceAccount
    name: netflix-sa

roleRef:
  kind: Role
  name: netflix-role
  apiGroup: rbac.authorization.k8s.io
```
Ikkada subjects → permissions receive chese identity. name: netflix-sa → mana ServiceAccount. roleRef → assign cheyyalsina Role netflix-role.
Flow:
Role
↓
get + list Pods
↓
RoleBinding
↓

netflix-sa
↓
Netflix Pod
So simple ga:
Role → Permissions define chestundi.
RoleBinding → Aa permissions evariki ivvalo chepthundi.

---

12. networkpolicy.yaml
networkpolicy.yaml Pods madhya **ye network traffic allow cheyyali, ye traffic restrict cheyyali** ani control cheyyadaniki use chestham. Example ga frontend Pods backend Pods ni access cheyyachu kani random Pods backend ni access cheyyakudadhu anukundam. Appudu NetworkPolicy use chestham. Idi Pods or containers create cheyyadu; existing Pods madhya network access rules apply chestundi.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy

metadata:
  name: netflix-backend-policy

spec:
  podSelector:
    matchLabels:
      app: backend

  policyTypes:
    - Ingress

  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend

      ports:
        - protocol: TCP
          port: 8080
```

Ikkada podSelector → app: backend unna Pods ki policy apply avtundi. from → app: frontend unna Pods nunchi traffic allow chestunnam. port: 8080 → backend port 8080 ki traffic allow chestunnam.
Flow:
Frontend Pod
↓
Backend Pod :8080
✓ Allowed

Other Pod
↓
Backend Pod
✗ Restricted
So simple ga:
NetworkPolicy → Pods madhya network traffic ni control chestundi.

---

13. values.yaml — Helm use chesthe
values.yaml normal Kubernetes object YAML kaadu; idi **Helm chart** lo customizable values maintain cheyyadaniki use chestham. Same application ni dev, staging, production environments lo deploy chestunnappudu replicas, image tag, service port, domain lanti values easy ga change cheyyachu. Helm ee values ni templates lo use chesi final Kubernetes YAML manifests generate chestundi.

```
replicaCount: 3

image:
  repository: mycompany/netflix
  tag: v1

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: true
  host: netflix.example.com
```

Ikkada replicaCount: 3 → Deployment replicas value. repository → Docker image repository. tag → image version. service.port → Service port. ingress.host → application domain.
Helm template example:

```
spec:
  replicas: {{ .Values.replicaCount }}

  template:
    spec:
      containers:
        - name: netflix
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Flow:

values.yaml
↓
Helm Template
↓
Final Kubernetes YAML
↓
Deployment / Service / Ingress
↓
Kubernetes

So simple ga:

values.yaml → Helm lo customizable values maintain cheyyadaniki.

Final Real-Time Flow

Application Source Code
↓
package.json
↓
Dockerfile
↓
Docker Image
↓
Container Registry
↓
deployment.yaml
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
↓
Netflix Application

Application ki surrounding Kubernetes configuration:

namespace.yaml
↓
Application environment/namespace

deployment.yaml
↓
Pods + Containers + Docker Image

service.yaml
↓
Pods ki networking

ingress.yaml
↓
Internet → Service

configmap.yaml
↓
Normal configuration

secret.yaml
↓

Passwords / API keys

hpa.yaml
↓
Automatic Pod scaling

pvc.yaml
↓
Persistent storage

serviceaccount.yaml
↓
Pod identity

role.yaml
↓
Permissions

rolebinding.yaml
↓
Permissions → ServiceAccount

networkpolicy.yaml
↓
Network traffic control

values.yaml
↓
Helm customization
Most important point: ee list lo deployment.yaml **main file** — endukante andulo containers: + image: untayi, replicas: untundi. Deployment → ReplicaSet → Pods create/manage avtayi, and **prathi Pod lo** containers **section prakaram container run avtundi; aa container lo** image: **lo ichina Docker image nunchi application run avtundi.**